

自注意力机制（上）

李宏毅《机器学习》2025 第四节：从可变长度输入到
Query-Key-Value



基于李宏毅老师课程整理

2026 年 6 月 8 日

视频信息

频道：李宏毅机器学习深度学习课程（Bilibili）

时长：约 28 分钟

链接：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=12>

目录

1 为什么 CNN 之后要讲 Self-attention	3
1.1 本章小结	4
2 向量集合从哪里来	4
2.1 文字：词可以变成向量	4
2.2 语音、社交网络与分子	5
2.3 本章小结	5
3 输出不只有一种形态	6
3.1 本章小结	7
4 逐位置 Fully Connected 的局限	7
4.1 本章小结	8
5 Self-attention：给每个位置一个带上下文的表示	9
5.1 从第一个输入产生第一个输出	9
5.2 本章小结	10
6 Query、Key 与 Attention Score	10
6.1 本章小结	11
7 Softmax 与 Value 加权求和	11
7.1 复杂度与下一讲伏笔	13
7.2 本章小结	13
8 总结与延伸	14
8.1 延伸阅读	14

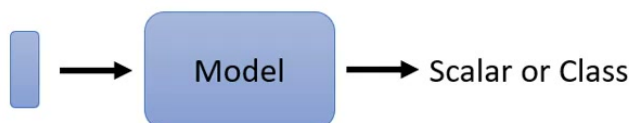
1 为什么 CNN 之后要讲 Self-attention

前面讲 CNN 时，输入通常被假设为固定尺寸的图像：每一张图可以整理成一个固定维度的张量或向量，然后交给模型做分类、回归或其他预测。Self-attention 要处理的情况更一般：输入不再只是一个向量，而是一组向量，而且这组向量的个数可以随样本改变。

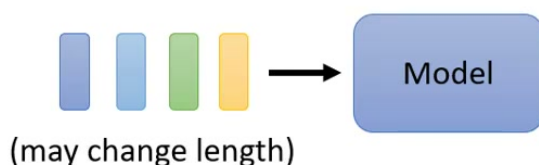
中文字幕 哔哩哔哩

Sophisticated Input

- Input is a **vector**



- Input is a **set of vectors** •



2

图 1: 从单一向量输入转向向量集合输入：模型必须能处理长度可变的一排向量。¹

把输入写成数学形式，就是

$$X = \{a^1, a^2, \dots, a^N\},$$

其中 a^i 是第 i 个输入向量， N 是这一笔资料里的向量个数。重点在于 N 不是全数据集固定的常数：一句话可以有 5 个词，也可以有 50 个词；一段语音可以切成几十个 frame，也可以切成几百个 frame；一个 graph 可以有少量节点，也可以有大量节点。

Self-attention 的问题意识

这堂课不是先假设某个漂亮公式，然后证明它有用；它的出发点是非常朴素的工程问题：当输入是一组数量不固定、彼此又可能有关联的向量时，fully connected network 逐个处理向量会漏掉上下文，而把整段硬塞进固定窗口又会遇到长度和参数量问题。Self-attention 正是为了解决“每个位置都要看见全局，但模型又要能处理可变长度”这一矛盾。

从这个角度看，self-attention 与 CNN 一样，都是一种 network architecture。CNN 把图像任务里的局部性、平移共享写进模型；self-attention 则把“序列或集合中的元素需要互相参考”写进

¹ 视频画面时间区间：约 00:40–00:55；字幕对应“假设我们说输入是一排向量”“输入的向量的数目会改变”。

模型。二者的共同点不是某个单独操作，而是都在通过结构假设限制模型家族，让模型用更合适的方式分配参数。

1.1 本章小结

Self-attention 要处理的核心输入是 variable-length vector set。它关心的不是单个向量本身够不够表达，而是向量之间的关系如何进入每个位置的表示。

2 向量集合从哪里来

2.1 文字：词可以变成向量

最直观的例子是文字。一个句子可以看成一串 token，每个 token 再被表示成一个向量。最简单的表示法是 one-hot encoding：为词表中每个词分配一个坐标，某个词出现时，对应坐标为 1，其余坐标为 0。

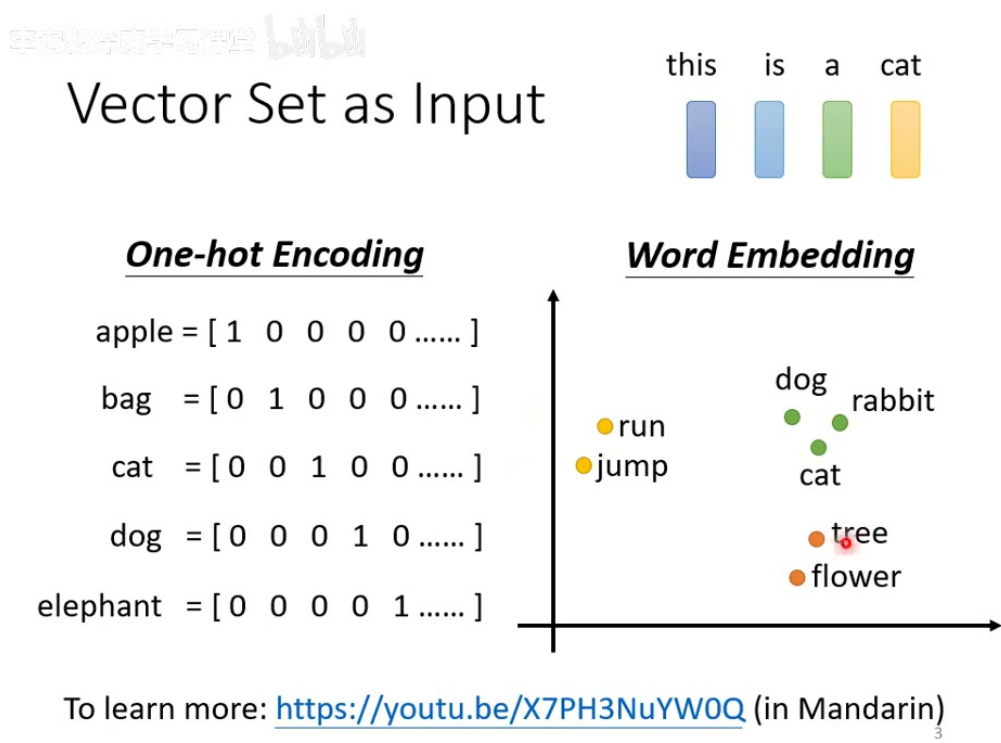


图 2: 词可以用 one-hot encoding 或 word embedding 表示成向量；一个句子自然就变成一组向量。
3

One-hot 的好处是定义简单，坏处也非常明显：每个词彼此正交，模型从这个表示本身看不出“dog”和“cat”比“dog”和“tree”更接近。因此实际系统常用 word embedding，把词映射到连续空间，让语义或使用场景相近的词在向量空间中较接近。课程此处不展开 embedding 的训练方法，而是借它说明：一段文字可以合理地变成一组向量。

³视频画面时间区间：约 01:45–03:00；字幕对应 “怎么把一个词汇表示成一个向量” “one hot encoding” “word embedding”。

2.2 语音、社交网络与分子

语音也可以转成 vector set。常见做法是把连续声音讯号切成短时间窗，例如每 25ms 取一个 window，再从中抽取 frame-level acoustic feature。每一小段语音对应一个向量，一整段语音就变成一串向量。

Graph 也可以看成向量集合。社交网络里每个节点代表一个人，节点的 profile、历史行为、关系统计等可以整理成向量；分子图里每个原子是节点，原子的元素种类、价键环境、局部结构也可以成为节点特征。这样，graph learning 中的一部分问题同样可以被放进 vector set 的框架。



<http://www.twword.com/wiki/%E5%88%86%E5%AD%90>

Vector Set as Input

- Graph is also a set of vectors (consider each **node** as a **vector**)



图 3: 分子可视为 graph，而 graph 又可以视为一组节点向量；这说明 vector set 不只出现在自然语言中。⁵

为什么这里强调“set”而不是“sequence”

文字和语音有明显顺序，所以常被称为 sequence；graph 的节点虽然也可以列成一排输入模型，但节点之间真正重要的是边和邻接关系，而不一定是列举顺序。因此老师用 vector set 这个更宽的说法：它允许输入是一批向量，也提醒我们后续还要处理“这些向量之间如何互相关联”的问题。

2.3 本章小结

文字、语音、社交网络、分子都可以统一成“若干个向量组成的一笔输入”。统一建模的好处是后面可以讨论同一种核心机制：每个向量如何从其他向量中取信息。

⁵视频画面时间区间：约 05:50–06:15；字幕对应“一个分子它也可以看作是一个 graph”。

3 输出不只有一种形态

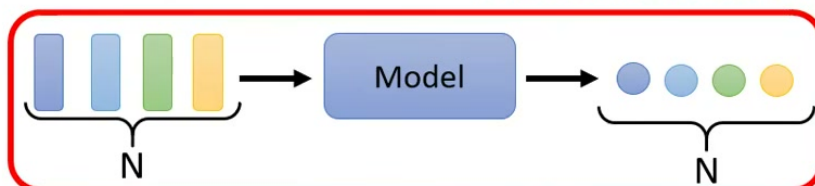
当输入是 N 个向量时，输出可以有几种不同形态。课程把它们分成三类：

1. **每个向量都有一个标签**：输入 N 个向量，输出 N 个预测，例如词性标注或语音 frame 的音标分类。
2. **整段输入只有一个标签**：输入 N 个向量，输出一个类别或数值，例如 sentiment analysis、speaker recognition 或分子性质预测。
3. **模型自行决定输出长度**：输入 N 个向量，输出 N' 个标签或 token，例如机器翻译、语音辨识等 seq2seq 问题。

What is the output?

- Each vector has a label.

focus of this lecture



- The whole sequence has a label.



- Model decides the number of labels itself.

seq2seq



图 4: 向量集合输入对应三类输出；本讲聚焦第一类：每个输入向量都要得到一个标签。⁶

本讲主要处理第一类，也就是 sequence labeling。它的形式可以写成

$$(a^1, \dots, a^N) \mapsto (y^1, \dots, y^N).$$

其中 y^i 是第 i 个输入位置的输出标签。典型例子是给句子 “I saw a saw” 做词性标注：第一个 saw 是动词，第二个 saw 是名词。模型若只看单词本身，就很难把两个完全一样的 token 分开。

⁶视频画面时间区间：约 10:45–12:10；字幕对应 “每一个 vector 都有一个 label” “整段只需要一个 label” “model decides the number of labels itself”。

同一个输入向量可能需要不同输出

Sequence labeling 的困难不在于“每个 token 单独分类”这件事本身，而在于同一个 token 在不同上下文里可能有不同角色。若模型把每个 a^i 独立送入同一个或不同的分类器，它看不到周围词，也就没有依据判断 saw 是动词还是名词。

3.1 本章小结

Self-attention 在这堂课里的切入点是 sequence labeling：输入和输出长度相同，但每个位置的预测必须依赖上下文，而不是只依赖当前位置的向量。

4 逐位置 Fully Connected 的局限

最直接的做法是把每个向量分别送进 fully connected network：

$$y^i = f(a^i).$$

这里 f 可以很复杂，但它只看 a^i 。如果两个位置的输入向量完全相同，且使用相同的 f ，输出就会相同。对“I saw a saw”这类例子来说，这显然不够。

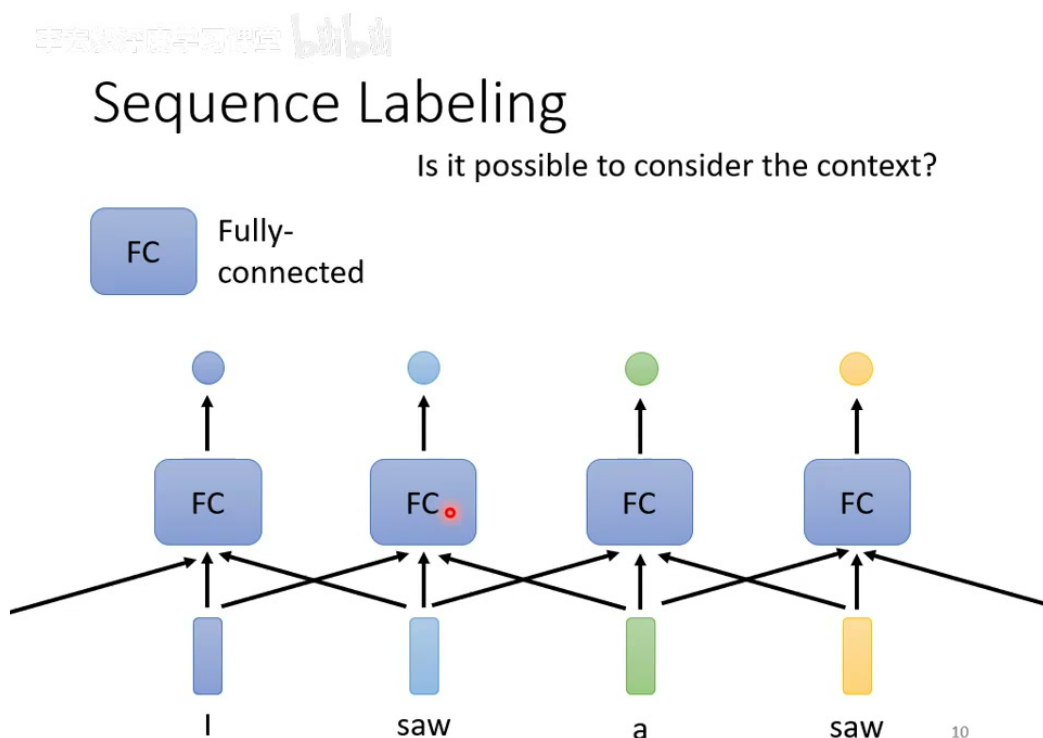


图 5: 逐位置 FC 可以给每个输入向量一个输出，但若不看上下文，就难以分辨两个 saw 的不同词性。⁸

⁸视频画面时间区间：约 13:30–14:30；字幕对应“你把这一个向量前后几个向量串起来”“作业二里面不是只看一个 frame”。

一个自然补救是开 window：预测第 i 个位置时，不只看 a^i ，还看它前后若干个向量。例如

$$y^i = f(a^{i-r}, \dots, a^i, \dots, a^{i+r}).$$

其中 r 是窗口半径。语音作业里常见的做法就是拼接前后几个 frame，让模型拥有局部上下文。

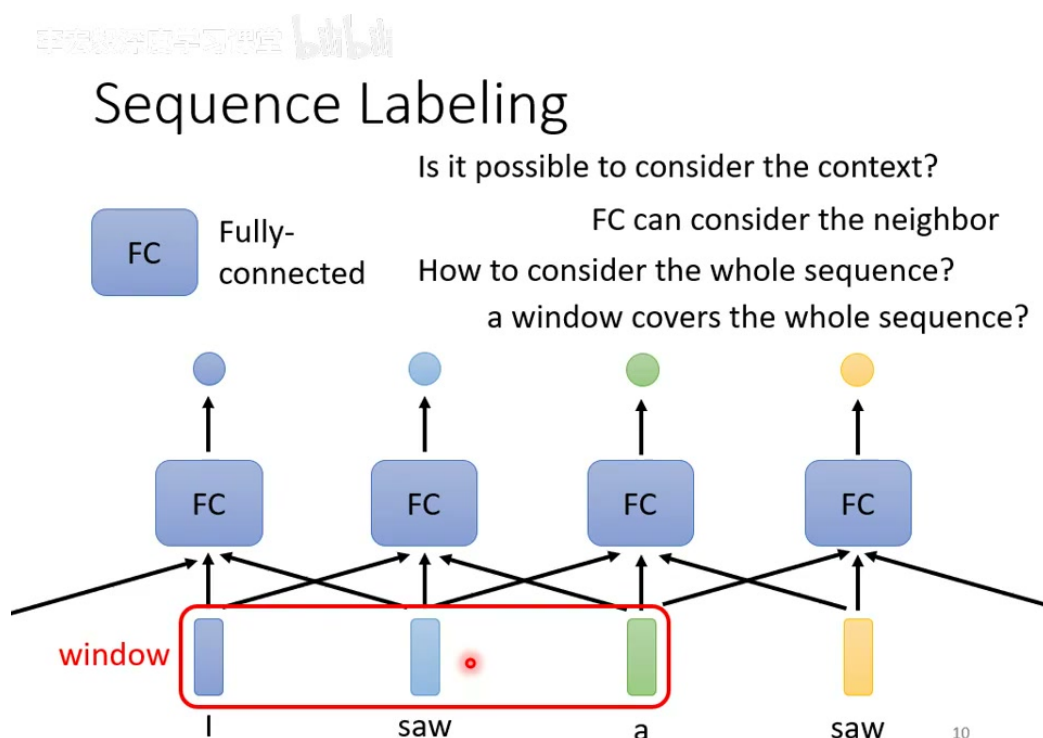


图 6: Window 方法能让 FC 看见邻近向量，但若想覆盖整段 sequence，就会遇到长度变化和参数量膨胀。¹⁰

Window 的问题有两层。第一，它预设“有用信息在附近”。但语言、语音、分子或图结构里，重要关系未必是局部的。第二，若想让 window 覆盖整个 sequence，就必须按训练集中最长序列来设定窗口；序列越长，拼接后的输入维度越高，第一层 FC 的参数也越多。

局部上下文不是全局上下文

Window 方法不是错，它在很多任务中有效；问题是它只能用固定半径近似上下文。Self-attention 的目标更强：对每一个位置，都让模型有机会参考整段输入中的任意位置，并且这种参考权重由数据自动决定。

4.1 本章小结

逐位置 FC 漏掉上下文；固定 window 只能补局部上下文，并且难以优雅处理可变长度。Self-attention 接下来要做的事，就是为每个位置构造一个已经吸收全局信息的新表示。

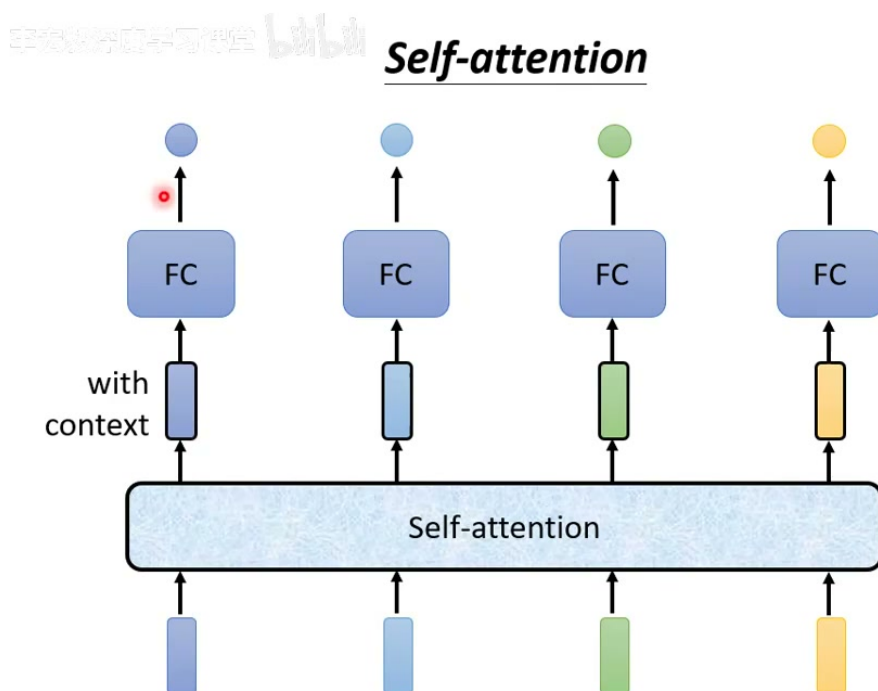
¹⁰视频画面时间区间：约 15:25–16:10；字幕对应“sequence 的长度有长有短”“开一个 window 比最长的 sequence 还要长”。

5 Self-attention: 给每个位置一个带上下文的表示

Self-attention 的输出仍然是一组向量:

$$(a^1, \dots, a^N) \mapsto (b^1, \dots, b^N).$$

但 b^i 不只是 a^i 的局部变换, 而是根据整个输入集合得到的 contextualized representation。直观地说, b^i 是“第 i 个位置在看完整段输入后形成的新向量”。



11

图 7: Self-attention 先把每个输入向量变成带上下文的向量, 再交给后续 FC 做逐位置预测。¹¹

课程中特别强调: self-attention 可以作为网络中的一个模块。它的输入可以是原始 embedding, 也可以是前面某个 hidden layer 的输出; 它的输出还可以继续交给 FC, 再接下一层 self-attention。实际模型常常就是 attention 与 feed-forward layer 交替堆叠。

“Attention is all you need” 的位置

老师提到著名论文 *Attention Is All You Need*, 是为了说明 self-attention 后来成为 Transformer 的核心部件。但这并不代表 self-attention 第一次出现于该论文; 更准确地说, 这篇论文把 attention 模块推到了一个极端: 不用 RNN/CNN 也能搭出强大的序列模型。

5.1 从第一个输入产生第一个输出

为了理解机制, 先只看第一个输出向量 b^1 。Self-attention 的第一步, 是让 a^1 去和序列中每个向量 a^j 比较相关性。相关性越高, a^j 对 b^1 的影响越大。

¹¹ 视频画面时间区间: 约 16:45–18:05; 字幕对应“考虑了整个句子以后才得到的资讯”“再把这个向量丢进 fully connected network”。

这个思想可以写成

$$b^1 = \sum_{j=1}^N \alpha'_{1,j} v^j.$$

符号含义如下：

- v^j 是从第 j 个输入向量抽出的 value，代表可被取用的信息内容；
- $\alpha'_{1,j}$ 是第 1 个位置对第 j 个位置的注意力权重；
- b^1 是所有 value 的加权和，权重越大，该位置信息进入 b^1 越多。

因此问题被拆成两个子问题：第一，怎么计算“相关性”分数；第二，怎么把相关性分数变成可解释的权重。

5.2 本章小结

Self-attention 的核心输出是 contextualized vector。每个 b^i 都由整段输入的信息混合而成，而混合比例来自该位置与其他位置的相关性。

6 Query、Key 与 Attention Score

最常见的相关性计算方法是 dot-product attention。对每个输入向量 a^i ，模型先用三个可学习矩阵产生 query、key、value：

$$q^i = W^q a^i, \quad k^i = W^k a^i, \quad v^i = W^v a^i.$$

符号含义如下：

- q^i 是 query，可以理解成第 i 个位置“想找什么信息”；
- k^i 是 key，可以理解成第 i 个位置“提供什么索引”；
- v^i 是 value，可以理解成第 i 个位置“真正可被汇入输出的内容”；
- W^q, W^k, W^v 是训练得到的参数矩阵。

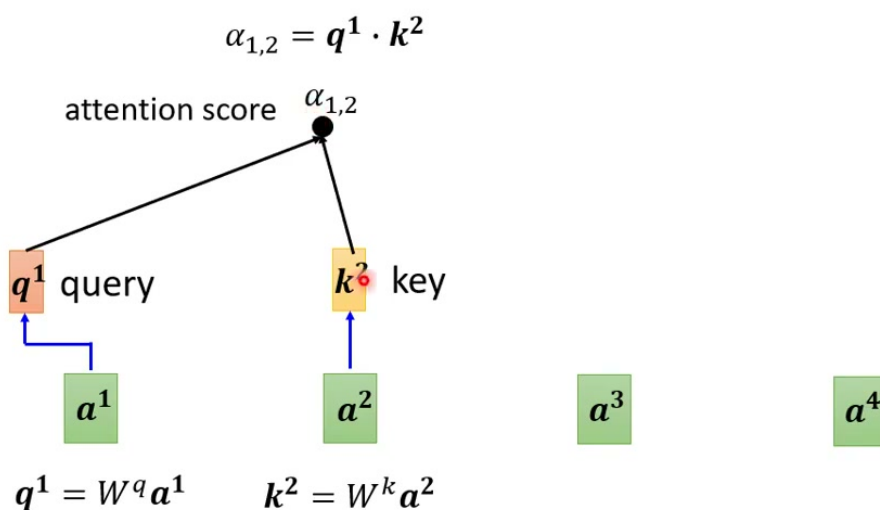
对第 1 个位置来说，它用 q^1 分别和所有 key 比较。第 1 个位置对第 2 个位置的原始 attention score 是

$$\alpha_{1,2} = q^1 \cdot k^2.$$

更一般地，

$$\alpha_{i,j} = q^i \cdot k^j.$$

Self-attention



16

图 8: Dot-product attention: 由 a^1 产生 query q^1 , 由 a^2 产生 key k^2 , 二者内积得到 attention score $\alpha_{1,2}$ 。¹³

为什么叫 query 和 key

这两个名字来自信息检索的直觉。Query 像是“我现在要找什么”，key 像是“我这里有什么索引”。当 query 与某个 key 的内积大，表示当前的位置更应该参考那个位置的信息。这个比喻不需要过度实体化：query/key/value 都只是由参数矩阵线性变换出来的向量，真正的含义由训练任务决定。

课程也提到 additive attention：把两个向量拼接起来，经过非线性变换再得到分数。这说明 attention score 并不只能用内积算；dot-product 的优势是形式简单、计算高效，也容易矩阵化。

6.1 本章小结

Attention score 的计算把“两个位置是否相关”交给可学习参数决定。Query 与 key 不是人工设计的规则，而是模型为了任务自动学出的比较空间。

7 Softmax 与 Value 加权求和

得到一排原始分数 $\alpha_{1,1}, \dots, \alpha_{1,N}$ 后，还需要把它们转换成权重。课程使用 softmax：

$$\alpha'_{1,i} = \frac{\exp(\alpha_{1,i})}{\sum_j \exp(\alpha_{1,j})}.$$

¹³视频画面时间区间：约 24:00–24:40；字幕对应“q one 跟 key k two 算 inner product 就得到阿尔法”。

这样做以后，所有权重为正，且加总为 1：

$$\sum_i \alpha'_{1,i} = 1.$$

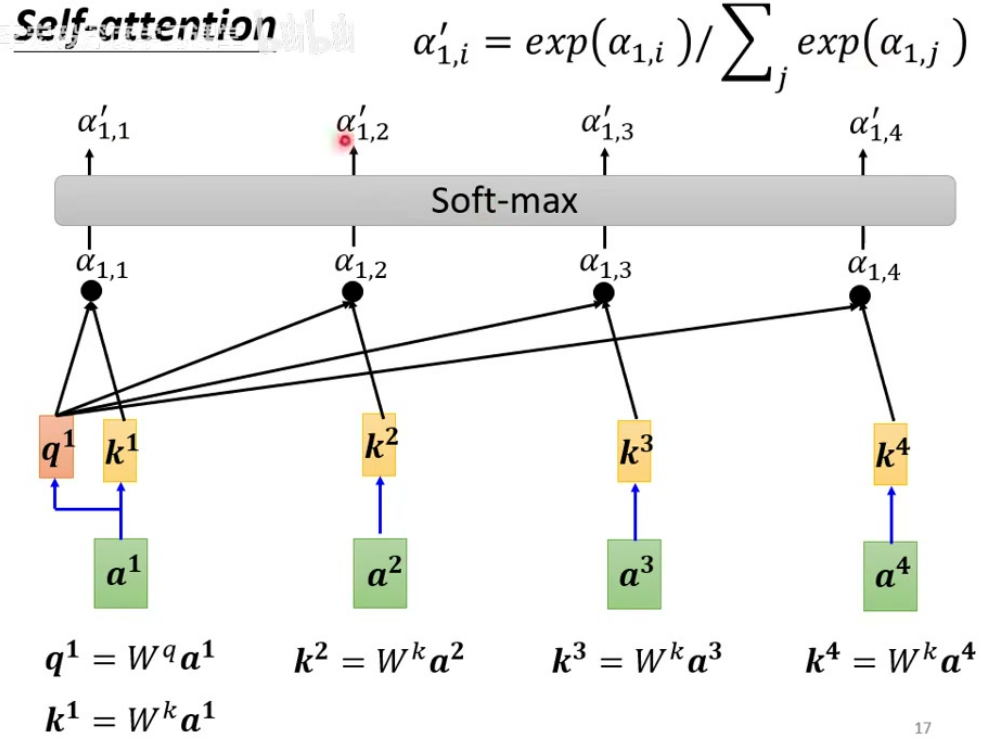


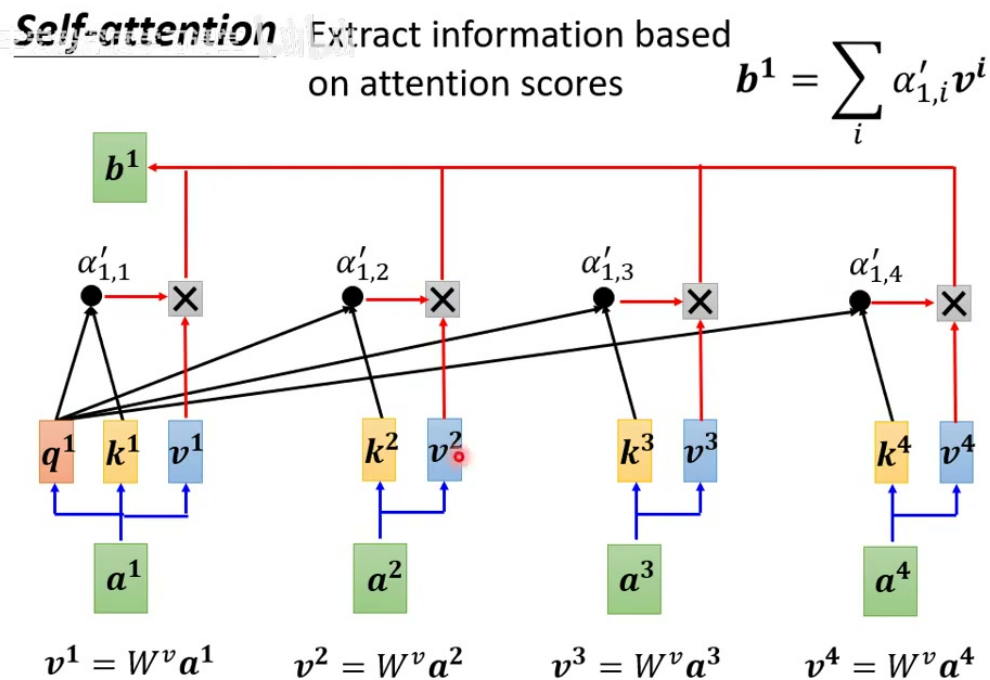
图 9: 对 attention scores 做 softmax，得到归一化后的注意力权重 $\alpha'_{1,i}$ 。¹⁴

最后一步是生成 value 并加权求和。每个输入向量 a^i 通过 W^v 得到 v^i ，再按注意力权重组合：

$$b^1 = \sum_i \alpha'_{1,i} v^i.$$

若 $\alpha'_{1,2}$ 很大，表示第 2 个位置对第 1 个位置的输出很重要，那么 v^2 就会在 b^1 中占更大比例。

¹⁴ 视频画面时间区间：约 25:40–26:20；字幕对应“计算出 a one 跟每一个向量的关联性以后，接下来会做一个 softmax”。



18

图 10: 根据 softmax 后的权重, 对所有 value 做加权和, 得到第一个输出向量 b^1 。¹⁵

把上述过程推广到每个位置, 就得到完整的单头 self-attention:

$$b^i = \sum_{j=1}^N \text{softmax}_j(q^i \cdot k^j) v^j.$$

其中 softmax_j 表示对固定的 query q^i , 在所有 j 位置的 score 上做归一化。

Self-attention 的一行压缩

对每个位置 i , 先用 q^i 去问 “我该参考谁”, 再用所有 k^j 回答 “每个位置和我有多相关”, softmax 把相关性变成权重, 最后用这些权重对 v^j 加权求和, 得到带上下文的 b^i 。

7.1 复杂度与下一讲伏笔

因为每个位置都要和每个位置比较, 单头 self-attention 至少会产生一个 $N \times N$ 的 score matrix。若序列很长, 计算和记忆体都会变重。这一点在后续讲 Transformer、长序列变体时会变得很重要。

本讲到这里还没有完整展开矩阵化写法、多头机制、位置资讯等内容。当前要抓住的是最小闭环: query-key 负责算关系, softmax 负责变权重, value 负责提供被汇入的新内容。

7.2 本章小结

Self-attention 的公式看起来复杂, 但流程很规整: 线性变换得到 Q/K/V, Q 与 K 算分数, softmax 得权重, 权重乘 V 得输出。

¹⁵ 视频画面时间区间: 约 27:25–28:15; 字幕对应 “把这边的每一个 V 都乘上 alpha prime 得到 b”。

8 总结与延伸

这节课的主线可以压缩成三个问题。

第一，为什么需要新的架构？因为许多任务的输入是长度可变的向量集合，且每个向量的正确解释依赖其他向量。逐位置 FC 看不到上下文，固定 window 又不能自然处理全局依赖与可变长度。

第二，self-attention 做了什么？它把每个输入位置转换成带上下文的新向量。对位置 i 来说，模型不是只处理 a^i ，而是让 a^i 与所有 a^j 建立可学习的相关性，再按相关性抽取全序列信息。

第三，Q/K/V 如何分工？Query 表达当前位置的检索需求，key 表达每个位置可被匹配的索引，value 表达真正被聚合的信息。注意力权重不是手写规则，而是由训练得到的参数矩阵和任务目标共同塑造。

不要把 attention 误解成“可解释性保证”

Attention weight 看起来像“模型正在看哪里”，但它首先是计算机制中的权重，不自动等同于人类语义解释。它能提示某些关系，却不保证完全解释模型决策。后续学习中应区分“机制上加权参考了哪些位置”和“因果上哪些位置导致了输出”。

8.1 延伸阅读

建议带着两个问题继续看下一讲：第一，如何把这里逐个位置的计算整理成矩阵乘法；第二，为什么单一组 Q/K/V 可能不够，需要 multi-head attention 同时学习多种关系。理解这两点以后，Transformer 的主体结构会自然许多。